



SQL Answer Key With Explanations

All 200 Questions

Explanations by John Pork

SQL ANSWER KEY

With Explanations — All 200 Questions

How to use this document: Each entry below repeats the question number and full question text from the 200-question bank, followed by the correct answer and a short explanation of the underlying SQL concept. Question numbers match the original question bank exactly, so you can cross-reference between the two documents.

***Reminder on Section 1:** Your screenshots showed an in-progress quiz with no answers selected, so there was no official graded key to pull from. These are my own determinations based on standard SQL — reliable for general understanding, but if a specific answer affects your exam score, verify it against your course's own material where possible.*

SECTION 1 — Your Original Questions

43 questions, in the same order as the question bank.

Q1. What does SQL stand for?

✓ **Correct Answer: Structured Query Language**

Explanation:

SQL expands to Structured Query Language. It's a language purpose-built for managing and querying data in relational databases — the word 'Structured' refers to the fact that data is organized into tables with defined columns and types.

Q2. Which of the following is used to select data from a database?

✓ **Correct Answer: SELECT**

Explanation:

SELECT is the statement used to read/retrieve data. INSERT adds new rows, JOIN combines tables (it's not a standalone data-retrieval command by itself), and DELETE removes rows. SELECT is always paired with FROM to specify the table.

Q3. Which SQL command is used to modify data in a table?

✓ **Correct Answer: UPDATE**

Explanation:

UPDATE changes existing values in existing rows. SELECT only reads data, DELETE removes rows entirely (rather than modifying values within them), and INSERT adds brand-new rows. UPDATE is always used with SET (to specify new values) and typically WHERE (to target specific rows).

Q4. Which of the following SQL statements will create a new table?

✓ **Correct Answer: CREATE TABLE**

Explanation:

CREATE TABLE is the standard DDL (Data Definition Language) statement for defining a new table, its columns, and their data types. The other options aren't valid SQL syntax.

Q5. Which SQL statement is used to delete data from a table?

✓ **Correct Answer: DELETE**

Explanation:

DELETE removes rows from a table based on a condition (via WHERE). Note: TRUNCATE also removes data but clears the *entire* table at once and can't use a WHERE clause — DELETE is the row-level, conditional option.

Q6. Which SQL clause is used to filter records?

✓ **Correct Answer: WHERE**

Explanation:

WHERE filters individual rows based on a condition, before any grouping happens. ORDER BY sorts results, GROUP BY groups rows into summary buckets, and HAVING filters *groups* (after GROUP BY), not individual rows.

Q7. Which of the following is the correct way to select all columns from a table named "Employees"?

✓ **Correct Answer: SELECT * FROM Employees**

Explanation:

SELECT * FROM Employees is standard syntax — the asterisk (*) means 'all columns.' 'GET', 'SHOW', and 'SELECT ALL' (as phrased here) aren't valid ways to request every column in standard SQL.

Q8. The SQL statement to find the average salary from a table would use which function?

✓ **Correct Answer: AVG()**

Explanation:

AVG() calculates the mean (average) of a numeric column. SUM() adds values together (a different result), and 'AVERAGE()' / 'MEAN()' are not real SQL functions — AVG() is the actual keyword.

Q9. What does the COUNT() function do in SQL?

✓ **Correct Answer: Counts the rows in a table**

Explanation:

COUNT() counts the number of rows that match a condition (or all rows, with COUNT(*)). It does not perform any arithmetic on the values themselves — MAX/MIN find highest/lowest values, and SUM adds numbers.

Q10. Which SQL function is used to add numeric values?

✓ **Correct Answer: SUM()**

Explanation:

SUM() adds up all values in a numeric column. COUNT() counts rows (not values), AVG() finds the mean, and MAX() finds the largest single value — none of those perform addition.

Q11. Which SQL function is used to get the highest value?

✓ **Correct Answer: MAX()**

Explanation:

MAX() returns the largest value in a column. MIN() does the opposite (smallest), COUNT() counts rows, and AVG() computes the mean — none of those identify the maximum.

Q12. Which of these functions is used to return the smallest value in SQL?

✓ **Correct Answer: MIN()**

Explanation:

MIN() returns the lowest value in a column. 'SMALL()', 'LOW()', and 'LEAST()' (as a single-column aggregate) are not the correct standard aggregate function — MIN() is.

Q13. Which SQL function is used to return the length of a string?

✓ **Correct Answer: LENGTH()**

Explanation:

LENGTH() returns the number of characters in a string. Note: some databases (like SQL Server) use LEN() instead — check your specific platform's docs, but LENGTH() is the ANSI-standard/most common form and the one this question is testing.

Q14. Which function in SQL returns the current date?

✓ **Correct Answer: NOW()**

Explanation:

NOW() returns the current date and time in most databases (particularly MySQL). Note: this can vary by platform — SQL Server uses GETDATE(), and some databases also support CURRENT_DATE. Since this question offers NOW() as an option, that's the expected answer here, but be aware of platform differences.

Q15. What is the default sorting order in SQL?

✓ **Correct Answer: Ascending**

Explanation:

Ascending (smallest/earliest to largest/latest) is the default when you use ORDER BY without specifying ASC or DESC. You have to explicitly add DESC to reverse it.

Q16. In SQL what is the purpose of the ORDER BY clause?

✓ **Correct Answer: Sorts data**

Explanation:

ORDER BY sorts the final result set by one or more columns, either ascending or descending. It does not filter, group, or join data — it purely controls the order rows are displayed in.

Q17. What is the purpose of the SQL ORDER BY clause?

✓ **Correct Answer: To sort the result set in ascending or descending order**

Explanation:

Same concept as above, phrased differently: ORDER BY's only job is sorting the result set (ascending or descending). It runs after filtering and grouping, right before the results are returned.

Q18. The SQL keyword BETWEEN is used for

✓ **Correct Answer: Checking ranges of values**

Explanation:

BETWEEN checks whether a value falls within a specified range (inclusive of both endpoints), e.g. WHERE age BETWEEN 18 AND 65. It's a shorthand for combining >= and <= with AND.

Q19. The SQL command SELECT * FROM employees WHERE salary BETWEEN 30000 AND 60000; does what?

✓ **Correct Answer: Selects employees with a salary between 30000 and 60000**

Explanation:

BETWEEN 30000 AND 60000 includes both endpoints and everything in between, so this returns every employee whose salary is 30000, 60000, or anywhere in that range — not just the two boundary values, and not employees outside the range.

Q20. What does the LIKE operator do in SQL?

✓ **Correct Answer: Checks for patterns in data**

Explanation:

LIKE performs pattern matching on text using wildcards (% for any number of characters, _ for exactly one character). It's used for partial/fuzzy text matches, e.g. WHERE name LIKE 'Jo%' finds names starting with 'Jo'.

Q21. In SQL the ____ clause is used to aggregate data by grouping

✓ **Correct Answer: GROUP BY**

Explanation:

GROUP BY organizes rows that share a value in specified column(s) into summary groups, so aggregate functions (SUM, COUNT, AVG, etc.) can be calculated per group instead of across the whole table.

Q22. What is the purpose of the HAVING clause?

✓ **Correct Answer: Filters groups of rows**

Explanation:

HAVING filters *groups* of rows after GROUP BY has run — typically used to filter based on an aggregate result, e.g. HAVING COUNT(*) > 5. This is different from WHERE, which filters individual rows before grouping.

Q23. What SQL clause is used to filter groups of data based on aggregate functions?

✓ **Correct Answer: HAVING**

Explanation:

Same as above — HAVING is specifically designed to work with aggregate functions (SUM, COUNT, AVG, etc.) because WHERE can't reference them (WHERE runs before aggregation happens).

Q24. Which SQL statement is used to retrieve unique values?

✓ **Correct Answer: SELECT DISTINCT**

Explanation:

SELECT DISTINCT removes duplicate rows from the result set, returning only unique combinations of the selected columns. 'SELECT DISTINCTIVE', 'SELECT UNIQUE', and 'SELECT DIFFERENT' are not valid SQL syntax.

Q25. In SQL what does the DISTINCT keyword do?

✓ **Correct Answer: It removes duplicates from the result set.**

Explanation:

DISTINCT removes duplicate rows from the query result, leaving only unique rows. It doesn't select 'distinct tables' or relate to column constraints — it operates purely on the output rows.

Q26. Which SQL clause is used to remove duplicate records?

✓ **Correct Answer: DISTINCT**

Explanation:

DISTINCT is the keyword for this. 'DELETE DUPLICATE' isn't valid SQL syntax for removing duplicates from a result set (though there are more complex techniques to permanently delete duplicate rows from a table — that's a different operation from DISTINCT).

Q27. What is the correct SQL statement to retrieve the top 5 records?

✓ **Correct Answer: SELECT TOP 5 * FROM table_name**

Explanation:

SELECT TOP 5 * FROM table_name is the correct syntax in SQL Server / MS Access style SQL. Note: this is dialect-specific — MySQL/PostgreSQL use 'SELECT * FROM table_name LIMIT 5' instead. Since this question offers TOP as an option, it's testing the SQL Server syntax specifically.

Q28. Which clause is used to limit the number of rows returned in a SQL query?

✓ **Correct Answer: All of the above**

Explanation:

All three (LIMIT, TOP, FETCH) accomplish row-limiting, just in different SQL dialects: LIMIT (MySQL/PostgreSQL), TOP (SQL Server), and FETCH (ANSI standard / Oracle / SQL Server). Since all three are valid depending on platform, 'All of the above' is correct here.

Q29. What type of JOIN returns all rows from both tables including rows with no match?

✓ **Correct Answer: FULL OUTER JOIN**

Explanation:

FULL OUTER JOIN combines LEFT JOIN and RIGHT JOIN — it returns every row from both tables, filling in NULLs where there's no match on either side. It's the most inclusive join type.

Q30. Which JOIN returns all records from the left table even if there are no matches in the right table?

✓ **Correct Answer: LEFT JOIN**

Explanation:

LEFT JOIN (also called LEFT OUTER JOIN) keeps every row from the left table regardless of whether a match exists in the right table — unmatched columns from the right table are filled with NULL.

Q31. To combine rows from two or more tables which SQL clause is used?

✓ **Correct Answer: JOIN**

Explanation:

Note on wording: JOIN is listed as correct here, but technically JOIN combines *columns* from related tables (matching on a key), while UNION combines *rows* by stacking similar result sets on top of each other. This question's phrasing ('combine rows from two or more tables') technically describes UNION more precisely — but JOIN is the intended/marked answer in your source material, so it's kept as-is. Worth flagging to your instructor if it affects your grade.

Q32. Which of the following constraints ensures that a column cannot have a NULL value?

✓ **Correct Answer: NOT NULL**

Explanation:

NOT NULL is a constraint that explicitly forbids empty/missing values in a column — any attempt to insert a row without a value for that column will fail. UNIQUE allows NULLs (just not duplicate non-NULL values), and DEFAULT just supplies a fallback value rather than forbidding NULL.

Q33. Which of the following SQL commands is used to delete a table from a database?

✓ **Correct Answer: DROP TABLE**

Explanation:

DROP TABLE permanently removes the table itself — including its structure, data, and all associated indexes/constraints. This is different from DELETE (which removes rows but keeps the table) or TRUNCATE (which empties the table but keeps its structure).

Q34. What does the SQL command TRUNCATE do?

✓ **Correct Answer: Deletes all rows in a table**

Explanation:

TRUNCATE removes ALL rows from a table at once, very quickly (it doesn't log individual row deletions the way DELETE does). It resets the table to empty but keeps the table structure intact — unlike DROP, which removes the table entirely.

Q35. Which of the following commands is used to remove all records from a table including all spaces allocated for the records?

✓ **Correct Answer: TRUNCATE**

Explanation:

TRUNCATE not only deletes all rows but also deallocates the storage space those rows were using, effectively resetting the table. DELETE removes rows but doesn't necessarily reclaim that storage space the same way.

Q36. Which SQL command is used to rename a column in a table?

✓ **Correct Answer: ALTER COLUMN**

Explanation:

Note on wording: This question lists 'ALTER COLUMN' as correct, but strictly speaking, ALTER COLUMN is typically used to change a column's *data type or constraints*, not its name. The full correct syntax for renaming is ALTER TABLE table_name RENAME COLUMN old_name TO new_name (see the next question for the fuller, more precise version of this same concept).

Q37. What is the correct syntax to rename a column in SQL?

✓ **Correct Answer: ALTER TABLE table_name RENAME COLUMN column_name TO new_name**

Explanation:

ALTER TABLE table_name RENAME COLUMN column_name TO new_name is the standard ANSI SQL syntax for renaming a column — it needs the full ALTER TABLE ... RENAME COLUMN structure, not just a bare 'RENAME COLUMN' statement.

Q38. How would you rename a table named "Orders" to "NewOrders" in SQL?

✓ **Correct Answer: ALTER TABLE Orders RENAME TO NewOrders;**

Explanation:

ALTER TABLE Orders RENAME TO NewOrders; is the standard syntax — you use ALTER TABLE to specify which table you're modifying, then RENAME TO followed by the new name.

Q39. Which of the following SQL statements is used to add a new column to an existing table?

✓ **Correct Answer: ALTER TABLE ... ADD**

Explanation:

ALTER TABLE ... ADD is the correct pattern, e.g. ALTER TABLE table_name ADD column_name datatype;. This modifies an existing table's structure to include a new column, as opposed to CREATE TABLE (which builds a table from scratch).

Q40. Which keyword is used to create a new database in SQL?

✓ **Correct Answer: CREATE DATABASE**

Explanation:

CREATE DATABASE is the standard DDL command for making a new database, e.g. CREATE DATABASE mydb;. It mirrors the pattern of CREATE TABLE, just at the database level instead of the table level.

Q41. How can you create an alias for a column in SQL?

✓ **Correct Answer: AS**

Explanation:

AS is the keyword used to rename a column (or table) for the duration of a query's output, e.g. SELECT salary AS monthly_pay FROM employees. It doesn't change the actual column name in the table — just how it's labeled in the results.

Q42. In SQL what is a view?

✓ **Correct Answer: A virtual table based on the result-set of a query**

Explanation:

A view is a virtual table defined by a stored SELECT query — it doesn't physically store data itself (usually), but instead runs its underlying query each time you access it, always reflecting current data from the real tables it's based on.

Q43. What will SELECT ROUND(123.456, 1); return?

✓ **Correct Answer: 123.5**

Explanation:

ROUND(123.456, 1) rounds to 1 decimal place. Looking at the second decimal digit (5) to decide rounding: 123.456 rounds to 123.5 (since the digit after the first decimal place, 5, rounds the 4 up to 5).

SECTION 2 — Additional Practice Questions

157 questions, organized by topic, matching the question bank's structure.

SQL Basics & SELECT Statements

Q44. Which SQL statement retrieves all rows and columns from a table?

✓ **Correct Answer: SELECT * FROM table**

Explanation:

SELECT * FROM table is the standard pattern — SELECT specifies you want to read data, * means 'every column,' and FROM names the table. No WHERE clause means no filtering, so every row comes back too.

Q45. Which keyword specifies which table to query?

✓ **Correct Answer: FROM**

Explanation:

FROM names the table (or tables) a query pulls data from. It always follows SELECT and comes before WHERE, GROUP BY, or ORDER BY in a query's structure.

Q46. Can you SELECT specific columns instead of all columns?

✓ **Correct Answer: Yes, list column names separated by commas**

Explanation:

Yes — instead of *, you list the exact column names you want, separated by commas, e.g. SELECT name, salary FROM employees. This is actually best practice, since selecting only needed columns is more efficient than pulling everything.

Q47. Which symbol selects all columns in a table?

✓ **Correct Answer: ***

Explanation:

The asterisk (*) is SQL's wildcard for 'all columns.' It's a shorthand so you don't have to type out every column name individually.

Q48. What does a semicolon (;) typically indicate in SQL?

✓ **Correct Answer: The end of a statement**

Explanation:

A semicolon marks the end of a SQL statement, similar to a period ending a sentence. It's required when running multiple statements in sequence, though many single-statement tools will run fine without it.

Q49. Are SQL keywords case-sensitive?

✓ **Correct Answer: Generally no, but conventionally written in uppercase**

Explanation:

Generally, SQL keywords (SELECT, FROM, WHERE, etc.) are NOT case-sensitive — 'select' and 'SELECT' work identically. However, writing them in uppercase is a strong convention that makes queries easier to read at a glance. (Note: table/column names CAN be case-sensitive depending on the database and OS.)

Q50. Which clause restricts which rows are returned before grouping?

✓ **Correct Answer: WHERE**

Explanation:

WHERE filters rows before any GROUP BY happens — it operates on the raw, ungrouped data. This is why WHERE can't reference aggregate functions like SUM() or COUNT() (those don't exist yet at the WHERE stage).

Q51. What is a NULL value in SQL?

✓ **Correct Answer: A missing or undefined value**

Explanation:

NULL represents missing, unknown, or inapplicable data — it is NOT the same as zero or an empty string. It's a special marker meaning 'no value was recorded here,' which is why comparing to NULL requires the special IS NULL operator instead of =.

Q52. Which operator checks if a field IS a missing value?

✓ **Correct Answer: IS NULL**

Explanation:

IS NULL is the correct operator. You cannot use = NULL because NULL isn't a value that can be equal to anything (including itself) — SQL requires the special IS NULL / IS NOT NULL syntax specifically for this check.

Q53. Which operator combines multiple conditions where all must be true?

✓ **Correct Answer: AND**

Explanation:

AND requires every condition in the chain to be true for a row to be included, e.g. WHERE age > 18 AND city = 'NYC' only returns rows matching BOTH conditions.

Q54. Which operator combines conditions where at least one must be true?

✓ **Correct Answer: OR**

Explanation:

OR requires just one condition (out of possibly several) to be true for a row to be included, e.g. WHERE city = 'NYC' OR city = 'LA' returns rows matching either city.

Q55. Which operator negates a condition?

✓ **Correct Answer: NOT**

Explanation:

NOT reverses/inverts a condition's result — e.g. WHERE NOT (age > 18) is equivalent to WHERE age <= 18. It flips TRUE to FALSE and vice versa.

Q56. Which operator checks membership in a list of values?

✓ **Correct Answer: IN**

Explanation:

IN checks if a value matches any item in a provided list, e.g. WHERE country IN ('USA', 'Canada', 'Mexico'). It's a cleaner shorthand than writing multiple OR conditions.

Q57. What does the wildcard '%' represent in a LIKE pattern?

✓ **Correct Answer: Zero or more characters**

Explanation:

% matches zero, one, or many characters in a LIKE pattern. For example, 'A%' matches any string starting with A, regardless of what (or how much) comes after.

Q58. What does the wildcard '_' represent in a LIKE pattern?

✓ **Correct Answer: Exactly one character**

Explanation:

_ (underscore) matches exactly one character — no more, no less. For example, 'A_' matches any two-character string starting with A, like 'AB' or 'A1', but not 'ABC'.

Q59. Which pattern finds names starting with 'A'?

✓ **Correct Answer: LIKE 'A%'**

Explanation:

LIKE 'A%' matches any string that begins with 'A', followed by zero or more of any characters. The % after A allows anything (or nothing) to follow.

Q60. What is the correct order of the core SQL clauses?

✓ **Correct Answer: SELECT, FROM, WHERE, ORDER BY**

Explanation:

SELECT, FROM, WHERE, ORDER BY is the correct written order. (Behind the scenes, the database actually *processes* FROM and WHERE before SELECT, but the way you *write* a query always follows this SELECT-FROM-WHERE-ORDER BY sequence.)

Q61. Which comparison operator checks for inequality in most SQL dialects?

✓ **Correct Answer: <>**

Explanation:

<> is the ANSI-standard 'not equal to' operator, supported across virtually all SQL databases. (Many databases, like MySQL, also accept != as an alternative, but <> is the most universally portable choice.)

Q62. How do you comment a single line in most SQL dialects?

✓ **Correct Answer: -- comment**

Explanation:

-- (two hyphens) starts a single-line comment in standard SQL — everything after it on that line is ignored by the database. (Multi-line comments use /* ... */ instead.)

Q63. Which clause sorts a result set by multiple columns?

✓ **Correct Answer: ORDER BY col1, col2**

Explanation:

ORDER BY col1, col2 sorts first by col1, then uses col2 as a tiebreaker for rows where col1 values are identical. You can mix ASC/DESC independently for each column too, e.g. ORDER BY col1 ASC, col2 DESC.

Filtering, Sorting & Pattern Matching

Q64. Which keyword sorts results from highest to lowest?

✓ **Correct Answer: DESC**

Explanation:

DESC (descending) sorts values from largest to smallest (or Z to A for text, latest to earliest for dates). You add it after the column name in ORDER BY, e.g. ORDER BY price DESC.

Q65. Which keyword explicitly sorts results from lowest to highest?

✓ **Correct Answer: ASC**

Explanation:

ASC (ascending) sorts smallest to largest / A to Z / earliest to latest. It's actually the default behavior even without writing it, but you can specify it explicitly for clarity.

Q66. Which clause checks if a value falls within a range (inclusive)?

✓ **Correct Answer: BETWEEN**

Explanation:

BETWEEN checks if a value falls within a specified range, and importantly, it INCLUDES both boundary values — e.g. BETWEEN 10 AND 20 includes 10 and 20 themselves, not just values strictly in between.

Q67. Is the BETWEEN operator's range inclusive or exclusive of the endpoints?

✓ **Correct Answer: Inclusive**

Explanation:

Inclusive — BETWEEN 10 AND 20 matches 10, 20, and everything in between. If you wanted to exclude the endpoints, you'd need to write it manually with > and < instead of BETWEEN.

Q68. Which SQL feature lets you skip a number of rows before returning results?

✓ **Correct Answer: OFFSET**

Explanation:

OFFSET tells the query to skip a specified number of rows before it starts returning results — commonly used for pagination, e.g. showing 'page 2' of results by skipping the first page's worth of rows.

Q69. Which clause is typically combined with OFFSET for pagination?

✓ **Correct Answer: LIMIT**

Explanation:

LIMIT (paired with OFFSET) is the classic pagination combo: LIMIT controls how many rows to return per 'page,' while OFFSET controls where to start. E.g. LIMIT 10 OFFSET 20 returns rows 21-30.

Q70. How would you find rows where a column is NOT NULL?

✓ **Correct Answer: WHERE column IS NOT NULL**

Explanation:

WHERE column IS NOT NULL is the correct syntax. As with IS NULL, you cannot use standard comparison operators (!=, <>) with NULL — SQL requires this dedicated IS NOT NULL phrasing.

Q71. Which clause would return only rows where quantity is either 5, 10, or 15?

✓ **Correct Answer: WHERE quantity IN (5, 10, 15)**

Explanation:

WHERE quantity IN (5, 10, 15) is the cleanest way to check membership in a specific list of values — functionally equivalent to writing quantity = 5 OR quantity = 10 OR quantity = 15, but more concise.

Q72. What does NULLS FIRST / NULLS LAST control?

✓ **Correct Answer: Where NULL values appear in sorted results**

Explanation:

This clause (supported in some databases like PostgreSQL and Oracle, added onto ORDER BY) explicitly controls whether NULL values appear at the beginning or end of sorted results, since NULL's position isn't always obvious/consistent across databases by default.

Q73. Which operator finds rows NOT matching a list of values?

✓ **Correct Answer: NOT IN**

Explanation:

NOT IN is the inverse of IN — it excludes rows whose value matches anything in the given list. (Caution in real use: NOT IN can behave unexpectedly if the list contains a NULL value, so many developers prefer NOT EXISTS for that reason — but for this question, NOT IN is the direct answer.)

Q74. Which clause combines a text search across multiple LIKE conditions?

✓ **Correct Answer: LIKE with OR**

Explanation:

Chaining multiple LIKE conditions with OR lets you match several different patterns, e.g. WHERE name LIKE 'A%' OR name LIKE 'B%' finds names starting with either A or B.

Q75. How would you match names containing 'an' anywhere in the string?

✓ **Correct Answer: LIKE '%an%'**

Explanation:

LIKE '%an%' uses % on both sides, meaning 'any characters, then an, then any characters' — so it matches 'an' appearing anywhere in the string (start, middle, or end), unlike 'an%' (only at the start) or '%an' (only at the end).

Q76. Which case-insensitive matching function/operator exists in some databases?

✓ **Correct Answer: ILIKE**

Explanation:

ILIKE (available in PostgreSQL) works exactly like LIKE but ignores upper/lowercase differences. Note: this is dialect-specific — not all databases support ILIKE; some achieve the same effect using LOWER()/UPPER() with regular LIKE instead.

Aggregate & Built-in Functions

Q77. Which aggregate function calculates the total of numeric values?

✓ **Correct Answer: SUM()**

Explanation:

SUM() adds together every value in a numeric column (or expression), returning a single total. It ignores NULL values in the calculation.

Q78. Which function counts all rows, including those with NULLs, when used as COUNT(*)?

✓ **Correct Answer: COUNT(*)**

Explanation:

COUNT(*) counts every row in the result set, regardless of whether individual columns contain NULL — it's counting rows themselves, not evaluating column values. This is different from COUNT(column_name), which does skip NULLs in that specific column.

Q79. Does COUNT(column_name) include NULL values in that column?

✓ **Correct Answer: No, NULLs are excluded**

Explanation:

No — COUNT(column_name) only counts rows where that specific column has a non-NULL value. This is the key difference from COUNT(*), which counts ALL rows regardless of NULLs.

Q80. Which function converts text to uppercase?

✓ **Correct Answer: UPPER()**

Explanation:

UPPER() converts all letters in a string to capital letters, e.g. UPPER('hello') returns 'HELLO'. It's commonly used for case-insensitive comparisons or standardizing display formatting.

Q81. Which function converts text to lowercase?

✓ **Correct Answer: LOWER()**

Explanation:

LOWER() converts all letters in a string to lowercase, e.g. LOWER('HELLO') returns 'hello'. Like UPPER(), it's often used to normalize text for comparisons.

Q82. Which function removes leading and trailing whitespace from a string?

✓ **Correct Answer: TRIM()**

Explanation:

TRIM() strips whitespace (spaces) from the beginning and end of a string, e.g. TRIM(' hello ') returns 'hello'. It doesn't affect spaces in the middle of the text.

Q83. Which function joins two or more strings together?

✓ **Correct Answer: CONCAT()**

Explanation:

CONCAT() combines (concatenates) multiple strings into one, e.g. CONCAT('Hello', ' ', 'World') returns 'Hello World'. Some databases also support the || operator as a shorthand for the same operation.

Q84. Which function rounds a number to the nearest whole value?

✓ **Correct Answer: ROUND()**

Explanation:

ROUND() rounds a number to a specified number of decimal places (or to the nearest whole number if no decimal place is given). Unlike FLOOR/CEIL, it rounds based on standard mathematical rounding rules (0.5 and up rounds up).

Q85. Which function always rounds a number down?

✓ **Correct Answer: FLOOR()**

Explanation:

FLOOR() always rounds down to the nearest integer, regardless of the decimal value — e.g. FLOOR(4.9) returns 4, and even FLOOR(4.1) returns 4. This is different from ROUND(), which would round 4.9 up to 5.

Q86. Which function always rounds a number up?

✓ **Correct Answer: CEIL()**

Explanation:

CEIL() (short for 'ceiling') always rounds up to the nearest integer, regardless of the decimal value — e.g. CEIL(4.1) returns 5. This is the opposite behavior of FLOOR().

Q87. Which function returns the absolute (non-negative) value of a number?

✓ **Correct Answer: ABS()**

Explanation:

ABS() strips the negative sign from a number, returning its distance from zero — e.g. ABS(-7) and ABS(7) both return 7. Useful for calculations where direction/sign doesn't matter, only magnitude.

Q88. Which function extracts a portion of a string starting at a given position?

✓ **Correct Answer: SUBSTRING()**

Explanation:

SUBSTRING() (sometimes SUBSTR()) pulls out a specific chunk of a string based on a starting position and length, e.g. SUBSTRING('Hello World', 1, 5) returns 'Hello'.

Q89. Which function returns the first non-NULL value from a list of expressions?

✓ **Correct Answer: COALESCE()**

Explanation:

COALESCE() checks a list of values in order and returns the first one that isn't NULL, e.g. COALESCE(NULL, NULL, 'fallback') returns 'fallback'. It's commonly used to supply default values when data might be missing.

Q90. Which function returns NULL if two expressions are equal?

✓ **Correct Answer: NULLIF()**

Explanation:

NULLIF() compares two values — if they're equal, it returns NULL; if they're different, it returns the first value. It's often used to avoid division-by-zero errors, e.g. NULLIF(denominator, 0).

Q91. Which function returns the current date and time together?

✓ **Correct Answer: CURRENT_TIMESTAMP**

Explanation:

CURRENT_TIMESTAMP returns both the current date AND time. This is different from CURRENT_DATE, which returns only the date portion (no time component).

Q92. Which function replaces occurrences of a substring within a string?

✓ **Correct Answer: REPLACE()**

Explanation:

REPLACE() finds every occurrence of a target substring within a string and swaps it for a new one, e.g. REPLACE('Hello World', 'World', 'SQL') returns 'Hello SQL'.

Q93. Which function returns a random value?

✓ **Correct Answer: Depends on the database**

Explanation:

This is genuinely dialect-dependent — MySQL uses RAND(), while some other databases use RANDOM() or other variations. Since the function name isn't standardized across all SQL platforms, 'Depends on the database' is the safest general answer.

Q94. Which function calculates the remainder of a division?

✓ **Correct Answer: MOD()**

Explanation:

MOD() (short for 'modulo') returns what's left over after dividing one number by another, e.g. MOD(10, 3) returns 1 (since $10 \div 3 = 3$ remainder 1). Some databases also support the % operator for the same operation.

Q95. Which function returns the position of a substring within a string?

✓ **Correct Answer: POSITION() / INSTR()**

Explanation:

POSITION() (ANSI standard) or INSTR() (common in MySQL/Oracle) both find where a substring first appears within a larger string, returning the character position as a number.

Q96. Which function converts a value to a different data type?

✓ **Correct Answer: CAST() / CONVERT()**

Explanation:

CAST() (ANSI standard) or CONVERT() (SQL Server/MySQL specific) both change a value from one data type to another, e.g. converting a text string '123' into an actual integer 123 for numeric operations.

Grouping & Aggregation

Q97. Which clause groups rows that share the same values into summary rows?

✓ **Correct Answer: GROUP BY**

Explanation:

GROUP BY collapses multiple rows sharing the same value(s) in specified column(s) into a single summary row, typically so an aggregate function (SUM, COUNT, AVG, etc.) can be calculated per group.

Q98. Can you use aggregate functions like SUM() without GROUP BY?

✓ **Correct Answer: Yes, it aggregates the entire result set**

Explanation:

Yes — without GROUP BY, an aggregate function like SUM() treats the entire result set as one single group, returning one overall total. GROUP BY is only needed when you want *separate* totals broken out by category.

Q99. Which clause filters groups after aggregation has occurred?

✓ **Correct Answer: HAVING**

Explanation:

HAVING filters the summarized groups created by GROUP BY, and it runs AFTER aggregation, so it can reference aggregate results directly (unlike WHERE, which runs before aggregation and can't).

Q100. Can HAVING reference an aggregate function directly?

✓ **Correct Answer: Yes**

Explanation:

Yes — this is actually HAVING's whole purpose. Since HAVING runs after GROUP BY and aggregation, it can filter based on values like COUNT(*), SUM(column), etc., e.g. HAVING SUM(sales) > 1000.

Q101. Can WHERE reference an aggregate function directly?

✓ **Correct Answer: No, aggregates aren't evaluated yet at WHERE stage**

Explanation:

No — WHERE filters rows BEFORE any grouping or aggregation happens, so aggregate results (like SUM or COUNT) don't exist yet at that stage of query processing. This is precisely why HAVING exists as a separate clause.

Q102. Which clause would you use to find departments with more than 5 employees?

✓ **Correct Answer: HAVING COUNT(*) > 5**

Explanation:

HAVING COUNT(*) > 5 is correct — you're filtering based on a count that only exists after grouping employees by department, so it must go in HAVING (not WHERE, which can't reference COUNT()).

Q103. When grouping by multiple columns, how are groups formed?

✓ **Correct Answer: By each unique combination of the listed columns**

Explanation:

GROUP BY col1, col2 creates a separate group for every UNIQUE combination of values across both columns — so if you group by (city, department), you'd get one row per distinct city+department pairing.

Q104. Do aggregate functions like SUM() and AVG() ignore NULL values?

✓ **Correct Answer: Yes, NULLs are excluded from the calculation**

Explanation:

Yes — SUM(), AVG(), MAX(), and MIN() all automatically skip NULL values in their calculations (they don't treat NULL as zero). Only COUNT(*) counts NULL rows; COUNT(column) also skips NULLs in that specific column.

Q105. Which function returns the number of distinct values in a column?

✓ **Correct Answer: COUNT(DISTINCT column)**

Explanation:

COUNT(DISTINCT column) counts only unique values in a column, ignoring repeats. For example, if a column has values [1, 1, 2, 3, 3, 3], COUNT(DISTINCT column) returns 3 (unique values: 1, 2, 3), while COUNT(column) would return 6.

Q106. What is the difference between COUNT(*) and COUNT(DISTINCT column)?

✓ **Correct Answer: COUNT(*) counts all rows; COUNT(DISTINCT) counts unique values**

Explanation:

COUNT(*) counts every row in the result, including duplicates. COUNT(DISTINCT column) counts only the unique values found in that specific column, collapsing duplicates down to one each.

JOINS & Combining Tables

Q107. What does INNER JOIN return?

✓ **Correct Answer: Only rows with matching values in both tables**

Explanation:

INNER JOIN returns only rows where a match exists in BOTH tables based on the join condition. Any row from either table that doesn't have a corresponding match in the other table is excluded entirely.

Q108. What does RIGHT JOIN return?

✓ **Correct Answer: All matched rows plus all rows from the right table**

Explanation:

RIGHT JOIN (RIGHT OUTER JOIN) keeps every row from the right table, plus any matching rows from the left table — unmatched columns from the left table are filled with NULL. It's the mirror image of LEFT JOIN.

Q109. What is a self-join?

✓ **Correct Answer: Joining a table to itself**

Explanation:

A self-join joins a table to itself, treating it as if it were two separate tables (using aliases to distinguish them). It's useful for comparing rows within the same table, e.g. finding employees who share the same manager.

Q110. What does a CROSS JOIN produce?

✓ **Correct Answer: Every combination of rows from both tables**

Explanation:

CROSS JOIN produces the Cartesian product — every possible combination of rows between the two tables, with no matching condition at all. If table A has M rows and table B has N rows, CROSS JOIN produces $M \times N$ rows.

Q111. If table A has 4 rows and table B has 3 rows, how many rows does CROSS JOIN produce?

✓ **Correct Answer: 12**

Explanation:

12 rows ($4 \times 3 = 12$). CROSS JOIN pairs every single row in table A with every single row in table B, with no filtering — so the result size is always the product of both table sizes.

Q112. Which keyword is used to join tables based on columns with the same name in both tables?

✓ **Correct Answer: NATURAL JOIN**

Explanation:

NATURAL JOIN automatically matches columns that share the same name across both tables, without you needing to explicitly specify the join condition with ON. (Use with some caution — it can behave unexpectedly if tables happen to share a column name you didn't intend to join on.)

Q113. Which clause specifies the condition for a JOIN?

✓ **Correct Answer: USING (column) or ON**

Explanation:

You can specify a join condition either with ON (most flexible — works for any condition, even across differently-named columns) or USING(column) (a shorthand specifically for when both tables share a column with the identical name).

Q114. Is it possible to join more than two tables in a single query?

✓ **Correct Answer: Yes, by chaining multiple JOIN clauses**

Explanation:

Yes — you can chain multiple JOIN clauses together in a single query to combine data from three, four, or more tables, as long as each join has its own valid condition connecting it to the tables already joined.

Q115. What is the main difference between JOIN and UNION?

✓ **Correct Answer: JOIN combines columns across tables; UNION stacks rows from similar result sets**

Explanation:

JOIN combines columns from related tables side-by-side (matching rows based on a key), producing wider results with more columns. UNION stacks rows from two similar result sets on top of each other, producing a taller result with the SAME columns, just more rows.

Q116. Do the result sets combined with UNION need the same number of columns?

✓ **Correct Answer: Yes, and compatible data types**

Explanation:

Yes — both queries being combined with UNION must return the same number of columns, and those columns need compatible/matching data types in the same order. You can't UNION a 3-column result with a 5-column result.

Q117. What does UNION ALL do differently from UNION?

✓ **Correct Answer: Keeps duplicate rows**

Explanation:

UNION ALL keeps every row from both result sets, including exact duplicates. Plain UNION automatically removes duplicate rows from the combined output. UNION ALL is also typically faster since it skips the duplicate-checking step.

Q118. Which set operation returns rows common to both result sets?

✓ **Correct Answer: INTERSECT**

Explanation:

INTERSECT returns only the rows that appear in BOTH result sets — essentially the overlap between them, similar to the intersection of two sets in math.

Q119. Which set operation returns rows in the first result set but not the second?

✓ **Correct Answer: EXCEPT / MINUS**

Explanation:

EXCEPT (ANSI standard, also called MINUS in Oracle) returns rows that exist in the first query's results but do NOT appear in the second query's results — think of it as 'subtracting' one result set from another.

Keys, Constraints & Table Design

Q120. What is a PRIMARY KEY?

✓ **Correct Answer: A column (or set of columns) that uniquely identifies each row**

Explanation:

A PRIMARY KEY is a column (or combination of columns) that uniquely identifies each row in a table. It automatically enforces both uniqueness (no duplicate values allowed) and NOT NULL (every row must have a value).

Q121. Can a table have more than one PRIMARY KEY column?

✓ **Correct Answer: Yes, as a composite key of multiple columns**

Explanation:

Yes — this is called a composite (or compound) primary key, where two or more columns TOGETHER form the unique identifier, even though neither column alone would be unique. A table can still only have ONE primary key overall, but that key can span multiple columns.

Q122. What is a FOREIGN KEY?

✓ **Correct Answer: A column that references the PRIMARY KEY of another table**

Explanation:

A FOREIGN KEY is a column in one table that references the PRIMARY KEY of another table, creating a relationship between the two. It's how relational databases link related data across separate tables (e.g. an 'orders' table's customer_id referencing the 'customers' table's id).

Q123. What does a UNIQUE constraint enforce?

✓ **Correct Answer: No duplicate values allowed in that column**

Explanation:

UNIQUE ensures that every value in a column is different from every other value in that same column — no duplicates allowed. Unlike PRIMARY KEY, a table can have multiple UNIQUE constraints on different columns.

Q124. Can a column with a UNIQUE constraint contain NULL values?

✓ **Correct Answer: Typically yes, since NULL is not considered equal to another NULL**

Explanation:

Typically yes — most databases allow one or more NULL values in a UNIQUE column, because NULL is treated as 'unknown' rather than a specific value that could duplicate another NULL. (This behavior can vary slightly by database, but it's the common standard.)

Q125. Can a PRIMARY KEY column contain NULL values?

✓ **Correct Answer: No, PRIMARY KEY implies NOT NULL and UNIQUE**

Explanation:

No — PRIMARY KEY automatically implies both UNIQUE and NOT NULL. Since a primary key's whole purpose is to reliably identify every row, allowing NULL (an 'unknown' value) would defeat that purpose.

Q126. What does a CHECK constraint do?

✓ **Correct Answer: Validates that column values meet a specific condition**

Explanation:

CHECK validates that any value entered into a column satisfies a specific condition you define, e.g. CHECK (age >= 18) would reject any row trying to insert an age below 18.

Q127. What does a DEFAULT constraint do?

✓ **Correct Answer: Assigns a value automatically when none is provided**

Explanation:

DEFAULT automatically supplies a pre-set value for a column when no value is explicitly provided during an INSERT — e.g. DEFAULT CURRENT_DATE could auto-fill today's date if the person adding the row doesn't specify one.

Q128. What is referential integrity?

✓ **Correct Answer: Ensuring foreign key values correspond to existing primary key values**

Explanation:

Referential integrity means foreign key values must always correspond to an EXISTING primary key value in the referenced table (or be NULL, if allowed) — you can't have an 'orphaned' foreign key pointing to a record that doesn't exist.

Q129. What does ON DELETE CASCADE do on a foreign key?

✓ **Correct Answer: Automatically deletes related child rows when the parent is deleted**

Explanation:

CASCADE automatically deletes related child rows whenever the parent row they reference is deleted — e.g. deleting a customer would also automatically delete all of that customer's orders, keeping the data consistent.

Q130. What does ON DELETE SET NULL do on a foreign key?

✓ **Correct Answer: Sets the foreign key column to NULL when the parent is deleted**

Explanation:

SET NULL sets the foreign key column to NULL (rather than deleting the child row) when the referenced parent row is deleted — useful when you want to keep the child record but acknowledge its original link is gone.

Q131. What is a composite key?

✓ **Correct Answer: A primary key formed from two or more columns together**

Explanation:

A composite key is a primary key made up of TWO OR MORE columns combined together, where the combination is unique even if no single column alone is. Common in linking/junction tables, e.g. a (student_id, course_id) pair in an enrollments table.

Q132. What is an auto-incrementing column typically used for?

✓ **Correct Answer: Automatically generating a unique sequential value, often for primary keys**

Explanation:

Auto-increment automatically generates a new, unique sequential number (usually starting at 1 and going up) each time a new row is inserted — commonly used for simple primary keys so you don't have to manually assign IDs yourself.

Q133. Which data type would best store a person's exact age in years?

✓ **Correct Answer: INT**

Explanation:

INT (integer) is the appropriate choice for a whole number like age — there's no need for decimal precision, and INT is efficient for simple numeric values like this.

Q134. Which data type is generally best suited for storing monetary values precisely?

✓ **Correct Answer: DECIMAL / NUMERIC**

Explanation:

DECIMAL (or NUMERIC) is specifically designed for exact precision with decimal values, which is critical for money — FLOAT can introduce tiny rounding errors due to how it stores numbers in binary, which is dangerous for financial calculations.

Q135. What is the key difference between CHAR and VARCHAR data types?

✓ **Correct Answer: CHAR is fixed-length; VARCHAR is variable-length**

Explanation:

CHAR is FIXED-length — it always reserves the same amount of space regardless of how much text you actually store (padding with spaces if needed). VARCHAR is VARIABLE-length — it only uses as much space as the actual text requires, up to a maximum you define.

Q136. Which data type would you use to store a true/false flag?

✓ **Correct Answer: BOOLEAN**

Explanation:

BOOLEAN is purpose-built for two-state true/false (or yes/no) values. (Note: some databases, like older MySQL versions, actually implement BOOLEAN as a TINYINT under the hood, but conceptually and syntactically, BOOLEAN is the right data type to reach for.)

Modifying Data & Table Structure (DML/DDDL)

Q137. Which statement adds new rows to a table?

✓ **Correct Answer: INSERT INTO**

Explanation:

INSERT INTO adds one or more brand-new rows into a table, e.g. INSERT INTO employees (name, salary) VALUES ('Alex', 50000). It's the only DML statement specifically for creating new rows (as opposed to modifying existing ones).

Q138. Which clause specifies which rows an UPDATE statement should affect?

✓ **Correct Answer: WHERE**

Explanation:

WHERE targets exactly which rows an UPDATE should modify, based on a condition — e.g. UPDATE employees SET salary = 55000 WHERE id = 3 only updates the one row matching that condition.

Q139. What happens if you run an UPDATE statement without a WHERE clause?

✓ **Correct Answer: It updates every row in the table**

Explanation:

Without WHERE, the UPDATE applies to EVERY row in the table — this is a common and often catastrophic mistake, since it's easy to accidentally overwrite all your data at once. Always double-check your WHERE clause before running UPDATE.

Q140. What happens if you run a DELETE statement without a WHERE clause?

✓ **Correct Answer: It deletes every row in the table**

Explanation:

Without WHERE, DELETE removes EVERY row from the table (though the table structure itself remains). This is functionally similar to TRUNCATE in outcome, though DELETE is typically slower and (in most databases) can be rolled back within a transaction, whereas TRUNCATE often cannot.

Q141. Which statement changes the structure of an existing table?

✓ **Correct Answer: ALTER TABLE**

Explanation:

ALTER TABLE modifies an existing table's structure — adding/removing columns, changing data types, renaming things, adding constraints, etc. — without needing to drop and recreate the whole table.

Q142. Which clause removes a column from an existing table?

✓ **Correct Answer: ALTER TABLE ... DROP COLUMN**

Explanation:

ALTER TABLE ... DROP COLUMN removes a specific column (and all its data) from an existing table, e.g. ALTER TABLE employees DROP COLUMN middle_name;

Q143. Which statement permanently removes an entire table and its structure?

✓ **Correct Answer: DROP TABLE**

Explanation:

DROP TABLE deletes the table completely — its data, its column definitions, its indexes, everything. This is different from TRUNCATE (which empties data but keeps the table structure) or DELETE (which removes specific rows).

Q144. Between DELETE and TRUNCATE, which is typically faster for clearing all rows?

✓ **Correct Answer: TRUNCATE**

Explanation:

TRUNCATE is generally much faster because it doesn't log each individual row deletion the way DELETE does — it essentially deallocates the whole data block at once, rather than removing rows one by one.

Q145. Can a DELETE statement be rolled back within a transaction?

✓ **Correct Answer: Yes, typically**

Explanation:

Yes, typically — DELETE is a logged operation in most databases, meaning if it's run inside a transaction that hasn't been committed yet, you can use ROLLBACK to undo it. (TRUNCATE's rollback behavior varies more by database — some allow it, some don't.)

Q146. Which command removes an entire database?

✓ **Correct Answer: DROP DATABASE**

Explanation:

DROP DATABASE deletes the entire database — every table, every piece of data, all of it, permanently. This is a database-level equivalent of DROP TABLE.

Q147. Which statement is used to insert values into specific named columns only?

✓ **Correct Answer: INSERT INTO table (col1, col2) VALUES (...)**

Explanation:

INSERT INTO table (col1, col2) VALUES (...) lets you specify exactly which columns you're providing values for, leaving other columns to use their default value (or NULL, if no default is set) instead of requiring you to supply every single column.

Q148. Can a single INSERT statement add multiple rows at once?

✓ **Correct Answer: Yes, using multiple value sets separated by commas**

Explanation:

Yes — you can provide multiple sets of values, separated by commas, all within one INSERT statement, e.g. INSERT INTO table (col1) VALUES (1), (2), (3); adds three rows in a single command, which is more efficient than three separate INSERT statements.

Views, Subqueries & Set Concepts

Q149. What is a subquery?

✓ **Correct Answer: A query nested inside another query**

Explanation:

A subquery is a SELECT query nested INSIDE another SQL statement (SELECT, WHERE, FROM, etc.), used to compute an intermediate result that the outer query then uses. It lets you break complex logic into layered steps.

Q150. Can a subquery appear in the WHERE clause?

✓ **Correct Answer: Yes**

Explanation:

Yes — this is one of the most common uses, e.g. WHERE salary > (SELECT AVG(salary) FROM employees) filters rows based on a value calculated by an inner query.

Q151. Can a subquery appear in the FROM clause (as a derived table)?

✓ **Correct Answer: Yes**

Explanation:

Yes — this is called a derived table. You can treat a subquery's result as if it were a temporary table, e.g. SELECT * FROM (SELECT ...) AS temp_table, and even join it with other tables.

Q152. Which operator checks whether a subquery returns any rows at all?

✓ **Correct Answer: EXISTS**

Explanation:

EXISTS checks purely whether a subquery returns ANY rows (true) or none (false) — it doesn't care about the actual data returned, just whether something matched. This makes it efficient since the database can stop as soon as it finds one matching row.

Q153. What does the EXISTS operator return?

✓ **Correct Answer: TRUE or FALSE depending on whether rows are returned**

Explanation:

EXISTS returns a simple TRUE or FALSE depending on whether the subquery it's checking returns at least one row. It's commonly used in WHERE clauses to filter based on the presence (or absence, with NOT EXISTS) of related data.

Q154. Can a subquery return more than one row when used with the IN operator?

✓ **Correct Answer: Yes**

Explanation:

Yes — IN is specifically designed to work with a subquery that returns multiple rows/values, checking if the outer query's value matches ANY of them. (This is different from using a subquery with =, which requires the subquery to return exactly one value.)

Q155. What is a correlated subquery?

✓ **Correct Answer: A subquery that references a column from the outer query**

Explanation:

A correlated subquery references a column from the OUTER query within its own WHERE clause, meaning it can't run independently — it has to be re-evaluated once for every row processed by the outer query, which can make it slower than an uncorrelated subquery.

Q156. What is a Common Table Expression (CTE)?

✓ **Correct Answer: A temporary, named result set defined with WITH**

Explanation:

A CTE is a temporary, named result set that you define at the start of a query (using WITH) and can then reference later in that same query, as if it were a regular table. It exists only for the duration of that one query.

Q157. Which keyword introduces a CTE?

✓ **Correct Answer: WITH**

Explanation:

WITH starts a CTE definition, e.g. WITH high_earners AS (SELECT * FROM employees WHERE salary > 100000) SELECT * FROM high_earners;

Q158. What is the main benefit of using a CTE over a subquery?

✓ **Correct Answer: Improved readability and reusability within the query**

Explanation:

CTEs primarily improve readability — they let you name and organize complex logic into clear, labeled steps instead of deeply nesting subqueries inside each other. They can also be referenced multiple times within the same query without repeating the logic.

Q159. Can a view be queried just like a regular table?

✓ **Correct Answer: Yes**

Explanation:

Yes — once a view is created, you can run SELECT statements against it exactly as you would against a real table, e.g. SELECT * FROM my_view WHERE ... This is the whole point of a view: it acts as a reusable, saved query.

Q160. Is data physically stored inside a standard (non-materialized) view?

✓ **Correct Answer: No, it stores the query definition and pulls live data**

Explanation:

No — a standard view just stores the underlying query definition. Every time you query the view, it re-runs that query against the live, current data in the real tables — meaning the view always reflects up-to-date data, but doesn't save any storage space or offer performance benefits on its own.

Q161. What is a materialized view?

✓ **Correct Answer: A view whose result is physically stored and periodically refreshed**

Explanation:

Unlike a standard view, a materialized view DOES physically store its query's result on disk, like a snapshot. It needs to be manually or automatically refreshed periodically to stay current, but it can offer much faster read performance since the data doesn't need to be recalculated every time.

Q162. Can every view be updated directly (e.g., via UPDATE)?

✓ **Correct Answer: Not always — it depends on the view's complexity**

Explanation:

Not always — simple views (based on a single table, with no aggregates, joins, or complex logic) are often updatable. More complex views (with joins, GROUP BY, DISTINCT, etc.) usually cannot be updated directly, since it's ambiguous which underlying table/row the change should apply to.

Indexes, Performance & Transactions

Q163. What is the main purpose of a database index?

✓ **Correct Answer: To speed up data retrieval**

Explanation:

An index speeds up data retrieval by creating a separate, sorted structure that lets the database find specific rows quickly, without scanning the entire table row by row. It works similarly to an index at the back of a book.

Q164. Does adding an index typically speed up or slow down INSERT/UPDATE operations?

✓ **Correct Answer: Slow down slightly, since indexes must also be updated**

Explanation:

It slows them down slightly — every time you insert or update data, the database also has to update the index(es) tracking that data, which adds a small amount of extra work. This is the tradeoff: indexes speed up reads but add overhead to writes.

Q165. How many clustered indexes can a single table generally have?

✓ **Correct Answer: One**

Explanation:

Only one — a clustered index physically determines the order in which data is stored on disk, and a table's data can only be physically sorted one way at a time. (You CAN have multiple non-clustered indexes on the same table, though.)

Q166. What does EXPLAIN (or EXPLAIN PLAN) show for a query?

✓ **Correct Answer: How the database intends to execute the query**

Explanation:

EXPLAIN shows the database's intended execution plan for a query — which indexes it plans to use, in what order it plans to join tables, estimated row counts, etc. It's a diagnostic tool for understanding and optimizing query performance, not for viewing the actual query results.

Q167. What is a database transaction?

✓ **Correct Answer: A sequence of operations treated as a single unit of work**

Explanation:

A transaction is a group of one or more SQL operations treated as a single, indivisible unit of work — either ALL of them succeed and get saved together (COMMIT), or if something goes wrong, NONE of them are saved (ROLLBACK). This ensures data stays consistent even if something fails partway through.

Q168. Which statement saves all changes made during a transaction?

✓ **Correct Answer: COMMIT**

Explanation:

COMMIT permanently saves every change made during the current transaction, making them visible to other users/sessions and impossible to roll back afterward.

Q169. Which statement undoes changes made during an uncommitted transaction?

✓ **Correct Answer: ROLLBACK**

Explanation:

ROLLBACK undoes/reverses all changes made since the transaction began (or since the last COMMIT), as if those operations never happened. It only works before a COMMIT has been issued.

Q170. What does the 'A' in the ACID properties of transactions stand for?

✓ **Correct Answer: Atomicity**

Explanation:

Atomicity — meaning a transaction is treated as one indivisible unit: either every operation within it succeeds, or none of them take effect at all (there's no partial/half-completed state).

Q171. What does the 'D' in ACID stand for?

✓ **Correct Answer: Durability**

Explanation:

Durability — meaning once a transaction is committed, its changes are permanently saved, even if the system crashes or loses power immediately afterward. Committed data won't be lost.

Q172. What is a deadlock in a database?

✓ **Correct Answer: A situation where two transactions block each other indefinitely**

Explanation:

A deadlock occurs when two (or more) transactions are each waiting on a resource the other one is holding, creating a circular block where neither can proceed — like two people trying to pass each other in a narrow hallway, each waiting for the other to move first. Databases typically detect this and forcibly cancel one of the transactions to break the standoff.

Q173. What is the purpose of GRANT in SQL?

✓ **Correct Answer: To assign permissions to a user**

Explanation:

GRANT gives a specific user (or role) permission to perform certain actions — like SELECT, INSERT, UPDATE, or DELETE — on a particular table, view, or database object.

Q174. What is the purpose of REVOKE in SQL?

✓ **Correct Answer: To remove previously granted permissions**

Explanation:

REVOKE removes permissions that were previously given via GRANT, taking away a user's ability to perform certain actions on a database object.

Q175. What does a parameterized query primarily help prevent?

✓ **Correct Answer: SQL injection attacks**

Explanation:

Parameterized queries help prevent SQL injection attacks — a security vulnerability where malicious input is crafted to manipulate a query's logic. By treating user input strictly as DATA (rather than executable SQL code), parameterized queries neutralize this risk.

Q176. What does normalization aim to reduce in a database?

✓ **Correct Answer: Data redundancy**

Explanation:

Normalization reduces data redundancy (the same piece of information being stored in multiple places) by organizing data into well-structured, related tables. This also helps prevent inconsistencies that can arise when redundant copies of data get out of sync with each other.

Q177. What does First Normal Form (1NF) require?

✓ **Correct Answer: No repeating groups or arrays in a column**

Explanation:

1NF requires that each column holds only a single, atomic value — no repeating groups, lists, or arrays crammed into one column/cell. For example, instead of one 'phone_numbers' column holding '555-1234, 555-5678', you'd need a separate row (or table) for each phone number.

Q178. What is denormalization typically used for?

✓ **Correct Answer: Improving read performance by allowing some redundancy**

Explanation:

Denormalization deliberately introduces some redundancy (going against strict normalization rules) specifically to improve READ performance — by pre-combining data that would otherwise require expensive joins, queries can run faster, at the cost of using more storage and needing more careful updates.

Q179. Which SQL statement would you use to see a query's estimated execution plan?

✓ **Correct Answer: EXPLAIN**

Explanation:

EXPLAIN shows how the database planned to execute a given query — which indexes it'll use, what order operations happen in, etc. — helping you diagnose slow queries and understand performance bottlenecks.

Q180. What is a window function primarily used for?

✓ **Correct Answer: Performing calculations across a set of related rows without collapsing them**

Explanation:

Window functions perform calculations ACROSS a set of related rows (a 'window') WITHOUT collapsing those rows into a single summary row, the way GROUP BY does. This lets you see both individual row detail AND aggregate-style calculations side by side.

Q181. Which window function assigns a unique sequential number to each row?

✓ **Correct Answer: ROW_NUMBER()**

Explanation:

ROW_NUMBER() assigns a unique, sequential integer (1, 2, 3...) to each row within its window/partition, with no ties — even if two rows have identical values, they still get different numbers based on their order.

Q182. What is the difference between RANK() and DENSE_RANK()?

✓ **Correct Answer: RANK() leaves gaps after ties; DENSE_RANK() does not**

Explanation:

Both assign rankings, and both give tied rows the same rank number — but RANK() then SKIPS the next rank number(s) to account for the tie (e.g. 1, 2, 2, 4), while DENSE_RANK() does NOT skip any numbers (e.g. 1, 2, 2, 3).

Q183. What does the PARTITION BY clause do within a window function?

✓ **Correct Answer: Divides rows into groups for the calculation, similar to GROUP BY but without collapsing rows**

Explanation:

PARTITION BY divides the rows into separate groups for the window function's calculation — similar in concept to GROUP BY, but critically, it does NOT collapse the rows into summary rows; every individual row is still shown, just with the window calculation applied per its group.

Q184. Which window function looks at the value from the previous row?

✓ **Correct Answer: LAG()**

Explanation:

LAG() retrieves a value from a PREVIOUS row (relative to the current row) within the same result set, based on a specified ordering — useful for things like calculating period-over-period change.

Q185. Which window function looks at the value from the next row?

✓ **Correct Answer: LEAD()**

Explanation:

LEAD() retrieves a value from the NEXT row (relative to the current row), the mirror-image counterpart to LAG(). Both are commonly used for comparing a row to its neighbors in a sequence.

Q186. What is a stored procedure?

✓ **Correct Answer: A precompiled, reusable set of SQL statements saved in the database**

Explanation:

A stored procedure is a saved, precompiled block of SQL code (which can include logic like loops and conditionals, depending on the database) stored directly in the database itself, so it can be reused by simply calling it, rather than rewriting the same logic repeatedly.

Q187. What is a trigger in SQL?

✓ **Correct Answer: A procedure that automatically runs in response to certain table events**

Explanation:

A trigger is a special kind of stored procedure that automatically executes in response to a specific event on a table — such as before/after an INSERT, UPDATE, or DELETE — without needing to be manually called.

Q188. Which SQL statement is used to invoke a stored procedure?

✓ **Correct Answer: CALL / EXECUTE**

Explanation:

CALL (ANSI standard) or EXECUTE / EXEC (common in SQL Server) both run a previously-created stored procedure, e.g. CALL my_procedure(param1, param2);

Q189. What is meant by 'schema' in a database context?

✓ **Correct Answer: The structure/organization of the database (tables, columns, relationships)**

Explanation:

A schema refers to the overall STRUCTURE of a database — its tables, columns, data types, relationships, and constraints — essentially the blueprint that defines how data is organized, as opposed to the actual data values stored within that structure.

Q190. Which SQL clause would you use to rename a column only for the current query's output?

✓ **Correct Answer: AS (column alias)**

Explanation:

AS creates a column alias — a temporary display name that only applies to that query's results, without altering the actual column name stored in the table itself, e.g. SELECT first_name AS fname FROM users;

Q191. What best describes a CASE expression in SQL?

✓ **Correct Answer: A way to write conditional (if/then/else) logic within a query**

Explanation:

CASE lets you write conditional (if/then/else-style) logic directly within a query — e.g. returning 'High' or 'Low' based on whether a value exceeds a threshold — similar to an if-statement in other programming languages, but usable inline within SELECT, WHERE, or ORDER BY.

Q192. Which SQL feature converts row values into separate columns (e.g., for a summary report)?

✓ **Correct Answer: PIVOT**

Explanation:

PIVOT transforms unique values from one column into multiple separate columns in the output, effectively rotating data from a 'tall' format into a 'wide' one — commonly used for creating summary/cross-tab reports.

Q193. Which SQL feature converts columns back into rows?

✓ **Correct Answer: UNPIVOT**

Explanation:

UNPIVOT does the reverse of PIVOT — it takes data spread across multiple columns and converts it back into rows, going from a 'wide' format back to a 'tall' one.

Q194. What does the term 'ad-hoc query' refer to?

✓ **Correct Answer: A one-off query written for a specific, immediate need**

Explanation:

An ad-hoc query is a one-off query written for an immediate, specific need — as opposed to a saved, scheduled, or reusable query (like a stored procedure or a regularly-run report).

Q195. Which term describes copying and synchronizing data across multiple database servers?

✓ **Correct Answer: Replication**

Explanation:

Replication copies and keeps data synchronized across multiple database servers/instances, often used for redundancy (backup/failover purposes) or to distribute read load across several servers.

Q196. Which term describes splitting a large database across multiple servers by data ranges?

✓ **Correct Answer: Sharding**

Explanation:

Sharding splits (partitions) a large dataset across multiple separate database servers, typically based on some key (like a range of customer IDs or a geographic region), so that no single server has to hold the entire dataset — helping databases scale horizontally.

Q197. What isolation level allows a transaction to read uncommitted changes from another transaction?

✓ **Correct Answer: READ UNCOMMITTED**

Explanation:

READ UNCOMMITTED is the least strict isolation level — it allows what's called a 'dirty read,' where a transaction can see changes made by another transaction that hasn't even been committed yet (and might later be rolled back).

Q198. Which isolation level offers the strictest consistency guarantees?

✓ **Correct Answer: SERIALIZABLE**

Explanation:

SERIALIZABLE is the strictest isolation level — it essentially makes transactions behave as if they were run one at a time, in sequence, with no overlapping interference at all. This offers the strongest consistency but can reduce performance/concurrency.

Q199. What best describes 'dirty read' in database transactions?

✓ **Correct Answer: Reading uncommitted changes made by another transaction**

Explanation:

A dirty read happens when one transaction reads data that another transaction has changed but NOT YET committed — meaning that data could still be rolled back and never actually become 'real,' making the first transaction's read potentially inaccurate/unreliable.

Q200. Which SQL clause helps prevent the N+1 query problem in application code?

✓ **Correct Answer: Using a single JOIN instead of many separate queries per row**

Explanation:

Using a single JOIN to fetch related data all at once (instead of running one separate query PER row to fetch related details individually) avoids the N+1 problem — where an app runs 1 initial query, then N additional queries (one per result row), which becomes very inefficient as data grows.